Parcial Algoritmos y Programación II - 7541 Curso Calvo (2º instancia) 16 de noviembre de 2017

Padrón: **Apellido y Nombre:**

Punteros: APROBADO – DESAPROBADO **TDA:** APROBADO – DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){
    int *A, *C, *F;
    int **B;
    int H = 10 + ULTIMO_DIGITO_PADRON;

    A = new int[3];

    for (int i = 0; i < 3; i++) {
        A[i] = H + i;
    }

    C = new int;
    (*C) = A[1];
    F = A + 2;

    cout << (*F) << (*C) << A[0] << endl;

    B = new int*[3];
    B[0] = C;
    B[1] = F;
    B[2] = &H;

    cout << *B[1] << **B << *B[2] << endl;

    (*B[0]) = (*F) + 3;
    H++;
    A[2] = (*C) + 1;

    cout << *C << *B[1] <<  *B[2] << endl;

    F = C;
    C = A + 2;
    (**B) = A[0];
    A[2] = ** (B + 2);

    cout << H << (*C) << (*F) << endl;

    // liberar la memoria
    // ...

    return 0;
}
```

2. Implementar para la clase Cola<T> las primitivas, indicando pre y post condiciones:

```
void acolar(T elemento);
T desacolar();
```

3. Implementar el método **elegirPlanMasEconomico** de la clase **AsistenteComercial** a partir de las siguientes especificaciones:

```
class AsistenteComercial {

  public:

    /* post: selecciona de 'planesDisponibles' aquel PlanComercial de menor precio y que tenga los servicios indicados
     *       en 'serviciosRequeridos' como no adicionales.
     */
    PlanComercial* elegirPlanMasEconomico(Lista<PlanComercial*>* planesDisponibles, List<string>* serviciosRequeridos);

};
```

```
class PlanComercial {

  public:
    /* post: plan creado con el precion [$] indicado
     */
    PlanComercial(unsigned int precio);

    /* post: devuelve el precio [$] del plan.
     */
    unsigned int obtenerPrecio();

    /* post: devuelve los servicios que brinda el plan.
     */
    Lista<Servicios*>* obtenerServicios();

    ~PlanComercial();
};
```

```
class Servicio {

  public:
    /* post: inicializa el Servicio con el nombre indicado.
     */
    Servicio(string nombre, bool adicional);

    /* post: devuelve el nombre que identifica el Servicio.
     */
    string obtenerNombre();

    /* post: indica si el servicio es o no adicional.
     */
    bool esAdicional();

    ~Servicio();
};
```

4. Diseñar la especificación e implementar el **TDA Escudo**. Debe proveer operaciones para:

- Crearlo recibiendo como parámetro los puntos de resistencia que tiene disponibles.
- defenderAtaque: recibiendo como parámetro la cantidad de puntos del ataque, consume puntos de resistencia.
- estaDestruido: indica si el Escudo ya agotó sus puntos de resistencia.
- reparar: restaura la resistencia original, solamente si aún no fue destruido previamente.
- contarAtaquesSoportados: devuelve la cantidad de ataques que pudo defender sin ser destruido.

Los alumnos que tienen aprobado el parcialito de punteros/TDA no deben realizar el ejercicio ¼ respectivamente.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas

0
121110
121110
151611
111110

1
131211
131211
161712
121211

2
141312
141312
171813
131312

3
151413
151413
181914
141413

4
161514
161514
192015
151514

5
171615
171615
202116
161615

6
181716
181716
212217
171716

7
191817
191817
222318
181817

8
201918
201918
232419
191918

9
212019
212019
242520
202019

```
delete[] A
delete[] B
delete F
```

Los alumnos que tienen aprobado el parcialito de punteros/TDA no deben realizar el ejercicio ¼ respectivamente.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas